BÀI TẬP THỰC HÀNH REACT NATIVE

Chủ đề: CORE COMPONENTS & API — Ứng dụng BookStore Online

Mục tiêu tổng quát

- Thành thạo 10 component cốt lõi của React Native: `View`, `Text`, `Image`, `TextInput`, `Pressable`, `ScrollView`, `FlatList`, `Modal`, `Alert`, `ActivityIndicator`.
- Nắm chắc bộ prop quan trọng của từng component và biết chọn đúng component cho từng tình huống (ScrollView hay FlatList, Pressable hay Text có onPress, Modal hay Alert).
- Kết hợp component với state: ô tìm kiếm lọc dữ liệu tức thời, render có điều kiện, ba trạng thái màn hình loading / rỗng / có dữ liệu.
- Tiếp nối bài tập Layout tuần trước: layout đã tĩnh, tuần này làm cho nó **chạy** — bấm được, gõ được, cuộn mượt với danh sách dài.
- Chưa đi sâu vào gọi API thật hay điều hướng (navigation) — dữ liệu vẫn lấy từ `data.ts` cục bộ.
- Tài liệu tham khảo bắt buộc: reactnative.dev/docs/components-and-apis và reactnative.dev/docs/using-a-listview.

Cấu trúc chung mỗi buổi 2 buổi × 3 giờ

Giờ 1 — Component nền tảng: View, Text, Image

Lý thuyết trọng tâm

- Quy tắc số 1: mọi chữ hiển thị bắt buộc nằm trong `<Text>`. Không có chữ trần như HTML.
- `View` là khối chứa, không tự hiển thị gì; mặc định đã là flex container với `flexDirection: 'column'`.
- `Text`: `numberOfLines`, `ellipsizeMode`, `onPress`, `selectable`. Text lồng trong Text kế thừa style của Text cha.
- `Image`: phân biệt `source={{uri}}` (ảnh từ Internet, **bắt buộc** có width/height) và `source={require()}` (ảnh trong dự án).
- `resizeMode`: `cover` / `contain` / `stretch` / `center` — và `aspectRatio` để giữ tỉ lệ khi chiều rộng chạy theo phần trăm.
- `StyleSheet.create()` thay cho object inline: tạo một lần, không sinh object mới mỗi lần render.

Bài tập 1: BookCard đầy đủ — ảnh bìa thật, giá tiền, trạng thái hết hàng

Thay thế hoàn toàn phiên bản BookCard tĩnh của tuần 3. Thẻ gồm: ảnh bìa tải từ URL, tên sách tối đa 2 dòng, tên tác giả, giá tiền, và dòng chữ “Hết hàng” chỉ xuất hiện khi sách hết.

Yêu cầu kỹ thuật:

- Khai báo `interface BookCardProps` gồm `book: Book` và `onPress: (id: number) => void`. Không dùng `React.FC`.
- Ảnh bìa: `source={{ uri: book.cover }}`, `width: '100%'`, `aspectRatio: 3/4`, `borderRadius: 8`, `resizeMode="cover"`.
- Tên sách: `numberOfLines={2}` để thẻ không cao lệch nhau khi tên dài ngắn khác nhau.
- Render có điều kiện: `{!book.inStock && ...}` — chỉ hiện dòng “Hết hàng” khi `inStock` là `false`.
- Toàn bộ thẻ bọc trong `Pressable` gọi `onPress(book.id)`.
- Mọi style đặt trong `StyleSheet.create()` ở cuối file, không viết inline.

Gợi ý: nếu ảnh không hiện mà cũng không báo lỗi, gần như chắc chắn bạn quên đặt kích thước cho Image — ảnh từ URL không tự biết mình to bao nhiêu.

Bài tập 2: Dòng giá khuyến mãi bằng Text lồng nhau

Hiển thị giá gốc bị gạch ngang và giá sau giảm nằm ngay cạnh nhau trên cùng một dòng, với hai màu và hai cỡ chữ khác nhau.

Yêu cầu kỹ thuật:

- Dùng một `<Text>` cha bọc hai `<Text>` con — không dùng View + flexDirection row.
- Giá gốc: `textDecorationLine: 'line-through'`, màu xám, cỡ nhỏ hơn.
- Giá sau giảm: màu đỏ/cam, `fontWeight: '700'`.
- Viết hàm `formatPrice(n: number): string` trả về dạng `"96.000đ"` (có dấu chấm ngăn cách hàng nghìn).
- So sánh: thử dựng lại cùng giao diện bằng View + flexDirection row, và ghi lại 1 điểm khác biệt bạn quan sát được khi chữ xuống dòng.

Gợi ý: `n.toLocaleString('vi-VN')` *làm được việc định dạng, nhưng hãy thử tự viết bằng biểu thức chính quy để hiểu chuyện gì đang xảy ra.*

Giờ 2 — Tương tác và danh sách: TextInput, Pressable, FlatList

Lý thuyết trọng tâm

- `TextInput` là controlled component: luôn đi cặp `value={state}` + `onChangeText={setState}`. `onChangeText` nhận thẳng chuỗi, không phải event.
- Bộ prop hay dùng của TextInput: `placeholder`, `placeholderTextColor`, `keyboardType`, `autoCapitalize`, `autoCorrect`, `returnKeyType`, `onSubmitEditing`.
- `Pressable`: `onPress`, `onLongPress`, `hitSlop`, `android_ripple`, và `style` dạng hàm `({pressed}) => [...]`.
- ScrollView render toàn bộ item cùng lúc; FlatList chỉ render phần đang hiển thị. Danh sách ngắn cố định → ScrollView. Danh sách dài hoặc chưa biết số lượng → FlatList.
- FlatList: `data`, `renderItem`, `keyExtractor`, `numColumns`, `columnWrapperStyle`, `ListHeaderComponent`, `ListEmptyComponent`, `ItemSeparatorComponent`, `contentContainerStyle`.
- Phân biệt `style` (khung nhìn) và `contentContainerStyle` (nội dung cuộn) — padding phải đặt ở cái thứ hai.

Bài tập 1: BookListScreen có ô tìm kiếm lọc tức thời

Dựng màn hình danh sách sách với một ô tìm kiếm phía trên. Mỗi ký tự gõ vào, danh sách bên dưới lọc lại ngay lập tức theo tên sách, không cần bấm nút.

Yêu cầu kỹ thuật:

- `const [query, setQuery] = useState("")` — khởi tạo chuỗi rỗng.
- Lọc: `BOOKS.filter(b => b.title.toLowerCase().includes(query.toLowerCase()))`. Giải thích trong comment vì sao cần `toLowerCase()` ở **cả hai** vế.
- TextInput: `autoCapitalize="none"`, `autoCorrect={false}`, `returnKeyType="search"`, `placeholderTextColor` đặt rõ ràng.
- Danh sách dùng `FlatList` với `keyExtractor={(item) => item.id.toString()}`.
- `ListEmptyComponent` hiển thị "Không tìm thấy sách nào phù hợp" khi kết quả rỗng.
- Hiển thị số kết quả ngay dưới ô tìm kiếm, ví dụ "Tìm thấy 7 cuốn".

Gợi ý: đặt biến `filtered` ở thân component, ngay trước `return` — không cần useEffect, cũng không cần state thứ hai. Mỗi lần state đổi, component chạy lại và filtered được tính lại tự động.

Bài tập 2: Chuyển lưới sách tuần 3 sang FlatList 2 cột

Lấy đúng BookGrid đã dựng bằng flexWrap ở tuần 3, viết lại bằng FlatList — so sánh hai cách.

Yêu cầu kỹ thuật:

- `numColumns={2}` và `columnWrapperStyle={{ justifyContent: 'space-between' }}`.
- `ListHeaderComponent` chứa CategoryChips — phần này cuộn theo danh sách.
- `ItemSeparatorComponent` tạo khoảng cách 12 giữa các hàng.
- `contentContainerStyle={{ padding: 16, paddingBottom: 100 }}` để nút giỏ nổi không che sách cuối.
- Mỗi ô vẫn giữ badge giảm giá `position: 'absolute'` của tuần 3, kiểm tra badge không bị lệch.
- Viết vào comment đầu file: 2 lý do FlatList tốt hơn flexWrap ở đây, và 1 tình huống flexWrap vẫn tốt hơn.

Gợi ý: nếu console báo "VirtualizedLists should never be nested inside plain ScrollViews", nghĩa là bạn còn giữ ScrollView bọc ngoài. Bỏ nó đi — phần đầu chuyển hết vào ListHeaderComponent.

Giờ 3 — Phản hồi người dùng: Modal, Alert, ActivityIndicator

Lý thuyết trọng tâm

- `Modal`: `visible`, `transparent`, `animationType` (`none`/`slide`/`fade`), `onRequestClose` — bắt buộc trên Android để xử lý nút Back vật lý.
- `Alert.alert(title, message, buttons, options)` — là API gọi như hàm, không viết trong JSX. Mỗi nút: `{ text, onPress, style: 'default'|'cancel'|'destructive' }`.
- `ActivityIndicator`: `size` (`'small'`/`'large'`), `color`, `animating`.
- Ba trạng thái của một màn hình có dữ liệu: đang tải → rỗng → có dữ liệu. Mỗi trạng thái phải có giao diện riêng, không để màn hình trắng.
- Chọn đúng công cụ: Alert cho xác nhận ngắn theo phong cách hệ điều hành; Modal khi cần giao diện tuỳ biến hoặc nội dung dài.

Bài tập 1: Modal xem nhanh chi tiết sách

Bấm giữ (long press) lên một thẻ sách để mở lớp phủ hiển thị ảnh lớn, tên đầy đủ, tác giả, mô tả và giá — không điều hướng sang màn hình mới.

Yêu cầu kỹ thuật:

- `const [preview, setPreview] = useState<Book | null>(null)` — dùng chính giá trị null làm cờ đóng/mở.
- `<Modal visible={preview !== null} transparent animationType="fade">`.
- `onRequestClose` đặt lại `setPreview(null)`.
- Nền mờ: một `Pressable` phủ toàn màn hình, `backgroundColor: 'rgba(0,0,0,0.55)'`, bấm vào nền thì đóng Modal.
- Hộp nội dung nằm giữa màn hình, `borderRadius: 16`, `padding: 20`, chiều rộng `'86%'`.
- Kích hoạt bằng `onLongPress` trên BookCard, `onPress` vẫn giữ chức năng cũ.

Gợi ý: bấm vào hộp nội dung không được làm đóng Modal. Bọc hộp trong một Pressable riêng và để trống onPress của nó, hoặc chặn sự kiện lan ra ngoài.

Bài tập 2: Alert xác nhận xoá và ActivityIndicator lúc khởi động

Hai phản hồi khác nhau cho hai tình huống khác nhau: xác nhận trước hành động nguy hiểm, và báo cho người dùng biết app đang xử lý.

Yêu cầu kỹ thuật:

- Nút xoá trên mỗi thẻ sách gọi `Alert.alert` với 2 nút: "Huỷ" (`style: 'cancel'`) và "Xoá" (`style: 'destructive'`).
- Chỉ khi bấm "Xoá" mới thực sự loại sách khỏi state: `setBooks(prev => prev.filter(b => b.id !== id))`.
- Khi mới vào màn hình, hiển thị `ActivityIndicator size="large"` trong 1.5 giây rồi mới hiện danh sách.
- Dùng `useEffect` với `setTimeout` và mảng phụ thuộc `[]`. Nhớ **dọn dẹp** timer trong hàm return của useEffect.
- Trong lúc loading, ô tìm kiếm phải bị vô hiệu hoá (`editable={false}`).

Gợi ý: quên `clearTimeout` *trong cleanup sẽ gây cảnh báo cập nhật state trên component đã unmount. Đây là lỗi rất hay gặp khi bắt đầu dùng useEffect có timer.*

Bài tập: Tổng hợp Giờ 1 + 2 + 3

Mục tiêu: ghép toàn bộ thành một màn hình **BookListScreen** hoàn chỉnh, đủ 9 ô Component Bingo. Màn hình có: Header cố định → ô tìm kiếm → dãy chips danh mục → lưới sách 2 cột có badge → nút giỏ nổi → Modal xem nhanh → Alert xác nhận → vòng xoay loading.

Việc cần làm

- Tạo project mới, copy `data.ts` và 5 component đã có từ tuần 3 (`Header`, `CategoryChips`, `BookGrid`, `DiscountBadge`, `FloatingCartButton`).
- Viết mới `components/BookCard.tsx` (Giờ 1) và `components/SearchBar.tsx` (Giờ 2).
- Viết `screens/BookListScreen.tsx` theo khung dưới đây, điền phần `....`

```
import React, { useState, useEffect } from 'react';
import {
  View, Text, TextInput, FlatList, Modal, Pressable,
  Alert, ActivityIndicator, StyleSheet,
} from 'react-native';
import { BookCard } from '../components/BookCard';
import { CategoryChips } from '../components/CategoryChips';
import { FloatingCartButton } from '../components/FloatingCartButton';
import { BOOKS } from '../data';
import { Book } from '../types/Book';

export function BookListScreen() {
  const [books,   setBooks]   = useState<Book[]>(BOOKS);
  const [query,   setQuery]   = useState('');
  const [loading, setLoading] = useState(true);
  const [preview, setPreview] = useState<Book | null>(null);
  const [cart,    setCart]    = useState(0);

  // 1. Vong xoay 1.5s khi vao man hinh -- nho cleanup timer
  useEffect(() => {
    ...
  }, []);

  // 2. Loc theo ten sach, khong phan biet hoa/thuong
  const filtered = ...;

  // 3. Alert xac nhan truoc khi xoa
  const confirmRemove = (id: number) => {
    ...
  };
```

```
  if (loading) {
    return (
      <View style={styles.center}>
        <ActivityIndicator size="large" color="#4338CA" />
      </View>
    );
  }

  return (
    <View style={styles.screen}>
      {/* 4. O tim kiem -- value + onChangeText */}
      ...

      {/* 5. FlatList 2 cot, header la CategoryChips */}
      <FlatList
        data={filtered}
        keyExtractor={...}
        renderItem={...}
        numColumns={2}
        columnWrapperStyle={{ justifyContent: 'space-between' }}
        ListHeaderComponent={...}
        ListEmptyComponent={...}
        contentContainerStyle={styles.content}
      />

      {/* 6. Nut gio noi -- NGOAI FlatList */}
      ...

      {/* 7. Modal xem nhanh */}
      <Modal visible={preview !== null} transparent animationType="fade"
             onRequestClose={() => setPreview(null)}>
        ...
      </Modal>
    </View>
  );
}

const styles = StyleSheet.create({
  screen:  { flex: 1, backgroundColor: '#F8FAFC' },
  center:  { flex: 1, alignItems: 'center', justifyContent: 'center' },
  content: { padding: 16, paddingBottom: 100 },
});
```

Kiểm tra bài làm đúng khi

☐ Vào màn hình thấy vòng xoay khoảng 1,5 giây rồi mới hiện danh sách; ô tìm kiếm bị khoá trong lúc đó.

☐ Gõ “har” vào ô tìm kiếm — danh sách lọc ngay, và tìm được cả “Harry Potter” lẫn “HARRY”.

☐ Xoá hết nội dung ô tìm kiếm rồi gõ chuỗi vô nghĩa — hiện dòng “Không tìm thấy sách nào phù hợp”.

☐ Bấm giữ một thẻ sách — Modal hiện lên; bấm nền mờ thì đóng, bấm vào hộp thì không đóng.

☐ Bấm nút xoá — Alert hiện đúng phong cách hệ điều hành, bấm “Huỷ” thì sách vẫn còn.

☐ Cuộn danh sách xuống — Header đứng yên, nút giỏ tròn luôn nổi ở góc, sách cuối không bị che.

☐ Đủ 9 ô Component Bingo: View, Text, Image, TextInput, FlatList, Pressable, Modal, Alert, ActivityIndicator.

☐ Console không còn cảnh báo nào về key hay VirtualizedLists.

☐ `npx tsc --noEmit` không báo lỗi nào.

Nộp bài: screenshot màn hình chạy trên điện thoại thật (Expo Go) kèm mã nguồn, đặt trong thư mục week_4 theo đúng quy ước các tuần trước.